

Sommario

Progetto per l'esame di Impianti di Elaborazione.
GitHub Link: <https://github.com/antsiri/Progetto-di-Impianti-di-Elaborazione->

Keywords: Capacity Test

INDICE

1	Introduction	1
2	Capacity Test	1
2.1	Setup esperimento	1
2.2	Descrizione dei test	1
2.3	Grafici e analisi	2
2.4	Analisi della Fairness e della Ripetibilità del Benchmark	2
3	PCA e Clustering	3
4	Workload Characterization	3
5	DoE	3
6	Regression	3
7	Benchmark	3
8	FFDA	3
9	RDB e Fault Tree	3

1. INTRODUCTION

In questa documentazione vengono spiegati e risolti problemi legati agli Impianti di Elaborazione.

2. CAPACITY TEST

2.1. Setup esperimento

Lo scopo del seguente esperimento p quello di utilizzare la **massima capacità** di un Web Server.
A tale scopo, è stata impostata una piattaforma sperimentale su **macchina virtuale** che ospita un sistema operativo **Linux Ubuntu 20.04**, il quale a sua volta ha in esecuzione un servizio **Apache2** che funge da gestore del web server.
Per simulare i processi di richieste di un determianto **carico** sul web server, vengono generate dei file di varia dimesione utilizzando il comando `allocate`. Le dimesioni variano da pochi kiloByte fino a cento MegaByte.
La **caratterizzazione della capacity** è un test che richiede un certo numero di queste risorse per valutare la stabilità del sistema a fronte di un crescente tasso di richieste.
Le specifiche tecniche della macchina virtuale su cui è ospitato il web server sono le seguenti:

Tabella 1. Caratteristiche della Macchina Virtuale (UTM)

Categoria	Parametro	Valore
Sistema	Architettura	x86_64
	Macchina Emulata	Standard PC (Q35 + ICH9, 2009)
Sistema	Sistema Operativo	Ubuntu 22.04.5 LTS
Hardware	Memoria RAM	10 GB
Hardware	Dimensione Disco	25,75 GB
Rete	Tipologia Rete	Shared Network (e1000)

Note: Dati rilevati dal gestore di macchine virtuali UTM su host Mac.

Per l'esecuzione dei test, è stato utilizzato **Apache JMeter 21.0.12**, eseguito sulla macchina Host con le seguenti specifiche:

Tabella 2. Caratteristiche della Macchina Host (MacBook Pro)

Categoria	Parametro	Valore
Hardware	Modello	MacBook Pro (16-inch, 2019)
Hardware	Processore	2,3 GHz 8-Core Intel Core i9
Hardware	Memoria RAM	16 GB 2667 MHz DDR4
Hardware	Scheda Grafica	AMD Radeon Pro 5500M 4 GB
Hardware	Archiviazione	1 TB (Macintosh HD)
Sistema	Sistema Operativo	macOS Tahoe (v26.6)

Note: Dati rilevati dalle impostazioni di sistema della macchina Host.

2.2. Descrizione dei test

Il test si compone di un singolo **Thread Group**. Il gruppo ha la seguente configurazione:

- `Number of Threads (users)`: 50
- `Ramp-up Period (seconds)`: 25
- `Duration (seconds)`: 300

*Corresponding author: ant.sirignano@studenti.unina.it (A. Sirignano).

*These authors contributed equally to this work.

This document class was prepared on VSCode and compiled with pdfTeX. No errors were found during the compilation process. Rho-class supports external editors, though additional setup may be required.

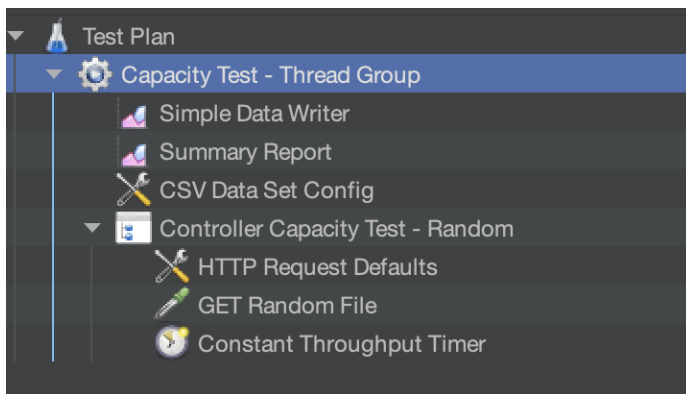


Figura 1. JMeter Test Plan.

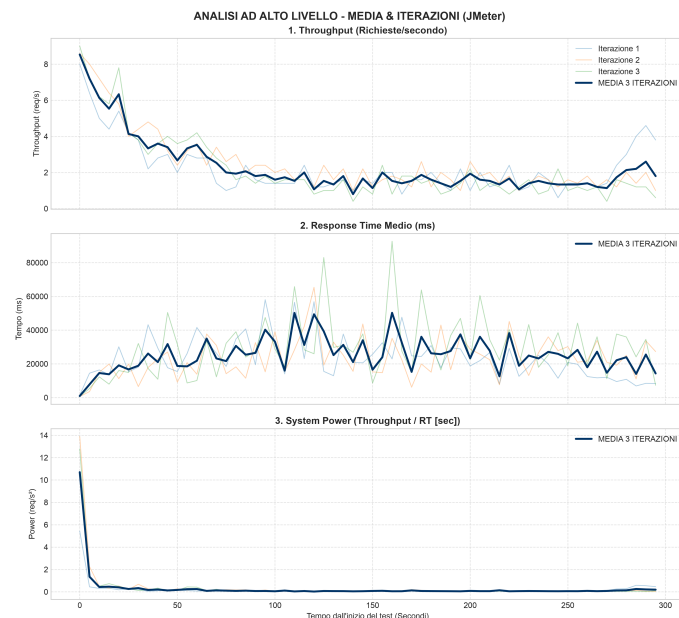
Nel test è stato utilizzato uno script che genera file di diverse dimensione sul server. Al fine di tenere traccia dei file generati, viene scritto un file `.csv` che contiene il path di ogni singolo file. Per questo nel test vi è un **CSV Data Set Config**, il quale legge ogni riga corrispondente al filepath di ogni file generato.

All'interno del gruppo è presente un **Random Controller**, in cui è inserita una richiesta HTTP di risorse. Il **Constant Throughput Timer** permette di regolare il numero di richieste da effettuare al minuto. Al fine di ottenere dei report esplicativi dell'esecuzione del test, è stato utilizzato un **Simple Data Writer**.

Per le informazioni a basso livello (analisi del bottleneck di prestazioni) è stato utilizzato il tool **vmstat**, fatto eseguire sul server.

Infine i dati ottenuti vengono processati attraverso uno script Python.

2.3. Grafici e analisi

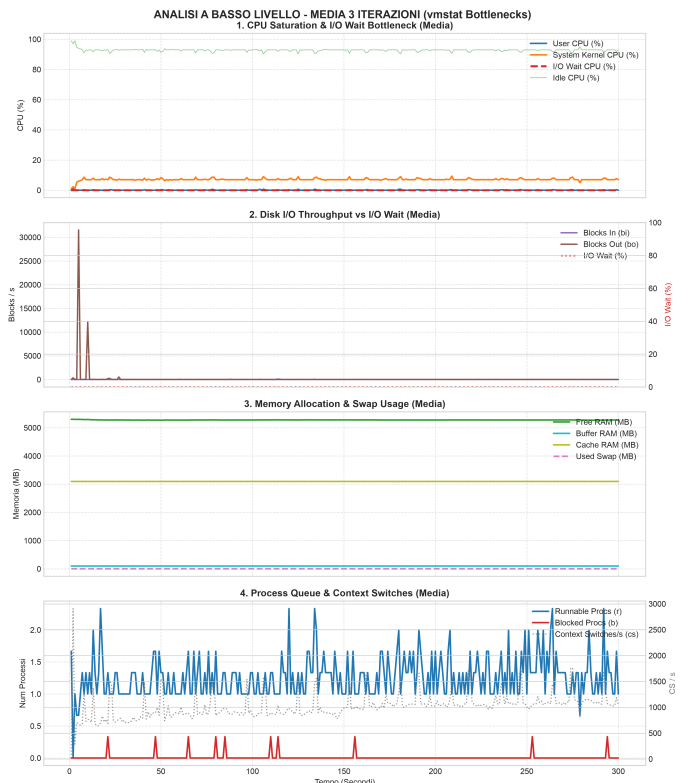


Eseguito lo script, sono stati ottenuti i seguenti grafici, che rappresentano rispettivamente il **throughput**, ovvero il tasso di richieste che il sistema ha potuto servire, il **response time**, ovvero la media dei tempi di risposta delle richieste per quel carico, e la **power**, che è un valore riassuntivo dei precedenti due parametri, ovvero

$$\text{Power} = \frac{\text{Throughput}}{\text{Response Time}} \quad (1)$$

Analizzando tale grafico si può notare che:

- Il Throughput crolla rapidamente da 8 a ~1.5 req/sec.
- Il Response Time esplode fino a 20-50 secondi.
- Il Power crolla a zero nei primi secondi: il sistema perde totalmente la sua efficienza di elaborazione.



Analisi dei bottleneck Analizzando i dati a basso livello, si vede che:

- La CPU è quasi totalmente in attiva (~92)
- La RAM libera è maggiore di 5GB.
- Il Disk I/O e I/O Wait sono piatti dopo il picco iniziale.

Questo fa pensare che il collo di bottiglia non è di tipo hardware. Si tratta invece di un bottleneck di configurazione software: i worker/thread di Apache sono saturi per via dei file grandi in download, mettendo le nuove richieste in coda senza che la CPU debba effettivamente lavorare.

2.4. Analisi della Fairness e della Ripetibilità del Benchmark

Per valutare l'equità nella ripartizione delle risorse di rete e la stabilità complessiva dei test di carico, è stato adottato l'**Indice di Fairness di Jain (Jain's Fairness Index)**.

La valutazione è stata condotta su due livelli: inter-iterazione (per verificare la riproducibilità e determinismo del test) e intra-iterazione (per rilevare eventuali fenomeni di starvation tra i thread/client concorrenti).

Dato un insieme di n entità concorrenti e definita con x_i la metrica di prestazione associata all'entità i -esima, l'indice Jain J è calcolato come:

$$J(x_1, x_2, \dots, x_n) = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2} \quad (2)$$

L'indice assume valori compresi nell'intervallo $[\frac{1}{n}, 1.0]$.

Poiché i test prevedono l'esecuzione di Thread Group Paralleli che richiedono risorse di dimensioni differenti, la metrica, x_i non misura

il semplice numero di richieste al secondo, che premierebbe fittiziamente i file di piccola dimensione, bensì la **banda passante occupata** espressa in MB/s:

$$x_i = \text{Bandwidth}_i = \frac{\text{Byte Total Trasferiti}_i}{1024^2 \cdot \Delta t} \quad (3)$$

L'indice di Jain inter-iterazione calcolato sia sul Throughput (0.9965) sia sulla Bandwidth (0.9968) è eccezionalmente prossimo al valore ideale di 1.0. Questo risultato attesta che il setup di test è perfettamente deterministico, stabile e riproducibile. Non sono state rilevate interferenze esterne (es. rumore di rete, processi di background sulla VM) che abbiano alterato i risultati tra un'esecuzione e l'altra.

Analizzando i dati all'interno della singola iterazione, l'indice di Jain calcolato sui thread scende sotto la soglia di ottimale (0.90), oscillando tra 0.7441 e 0.8163. Un valore di fairness significativamente inferiore a 1.0 all'interno della singola esecuzione dimostra l'insorgere di Thread Starvation. Sotto il carico massimo di 50 utenti concorrenti, le risorse di elaborazione del server web non vengono distribuite in modo uniforme: alcuni thread riescono ad accaparrarsi i socket di rete e completare più richieste, mentre altri rimangono bloccati nelle code del server per tempi prolungati (spiegando i tempi di risposta massimi fino a 200+ secondi registrati da JMeter).

3. PCA E CLUSTERING

4. WORKLOAD CHARACTERIZATION

5. DOE

6. REGRESSION

7. BENCHMARK

8. FFDA

9. RDB E FAULT TREE